

The stuff you save to get better at your job, and never look at again.

How I designed and built an AI tool for the things you save to learn from. The first version is for people studying product management, but nothing underneath it is tied to that.

Everyone saves things they mean to learn from. Almost nobody goes back. PM Content Vault takes the posts, PDFs, screenshots and chat exports sitting in five different apps, works out what each one is, and lets you ask questions of the whole pile, with every answer showing the saves it came from.

Role: Product & build, end to end · **Status:** Live

LIVE APP pm-content-vault.vercel.app · **SOURCE** github.com/RADHIKA2909/pm-content-vault
Read this online at pm-content-vault.vercel.app/prd-case-study

Try it without signing up: `demo@pmvault.app` / `PMVaultDemo2026!`

A demo account with one item of every supported type.

The problem

Anyone learning on their own builds up a pile of saved stuff. A framework from LinkedIn. A PDF dropped in a group chat. A screenshot. An article bookmarked at midnight. All saved on purpose, almost none of it opened again.

The easy answer is that people are messy and need better folders. I don't buy it. Saving isn't the broken part. Three things break after it.

It's scattered

LinkedIn, WhatsApp, bookmarks, a notes app. No one place to look, so looking anywhere feels pointless.

You can't judge it

A saved link is a title and a thumbnail. Finding out if it's worth ten minutes takes ten minutes.

You can't find it

You remember *"that post about handling stakeholder conflict"*. That's never a word that actually appears in it.

THE IDEA THIS PRODUCT IS BUILT ON

Storing things was never the problem. **Finding them and judging them** was. That's why every item gets a summary the moment it's saved, and why search came before tidying features.

Who it's for

Long term, anyone who saves things to learn from: exam takers, career switchers, people teaching themselves something new. The habit is the same everywhere.

The first group is **people learning product management**. The problem isn't special to them, but they share a vocabulary, a deadline and predictable content, which makes the categories worth having and the users easy to find.

WHY START NARROW WHEN THE PROBLEM IS WIDE

A fixed set of categories is what makes a saved pile scannable, and you can only pick good ones if you know the subject. Build for everyone on day one and you get vague categories, which pushes the sorting work back onto the user. That's the thing that already failed. **Opening it up later is a settings change, not a rebuild.**

Why build it when so much already exists

Lots of tools store saved content. None fix what was actually broken.

WHAT I COULD HAVE USED	WHY IT DOESN'T FIT
A notes app	Great storage, but sorting is your job, and that's the step people skip mid-scroll.
A read-later app	Built for a reading queue. It answers "what haven't I read", not "where's that thing about stakeholder conflict".
A general AI chatbot	Answers questions about a file you hand it, but every chat starts empty. Nothing builds up.
Platform search	Saved posts stay on the app that hosted them. Five searches instead of one, all keyword-only.

THE GAP THIS FILLS

Each option is good at one of three things: *take anything, understand it without being asked, answer across all of it with sources*. The value is doing all three to the same pile, and they depend on each other.

The journey, and where it still hurts

Four moments matter, and here is what still annoys me at each.

1

Spot something worth keeping

On a phone, inside LinkedIn or a group chat. You have a few seconds before the thing is gone.

Still annoying: copy → switch app → paste

The one step the MVP doesn't fix. It's first on the roadmap.

2

Save it and stop thinking

Paste, upload, or drop in a chat export. The app does the reading and sorting, and shows you what it got before anything is saved.

No filing, no folders, no naming

3

Come back days later, half-remembering

Browse the Library, where each card shows a gist, a type and a read time. Or just ask a question in plain English.

Search by meaning, not keywords

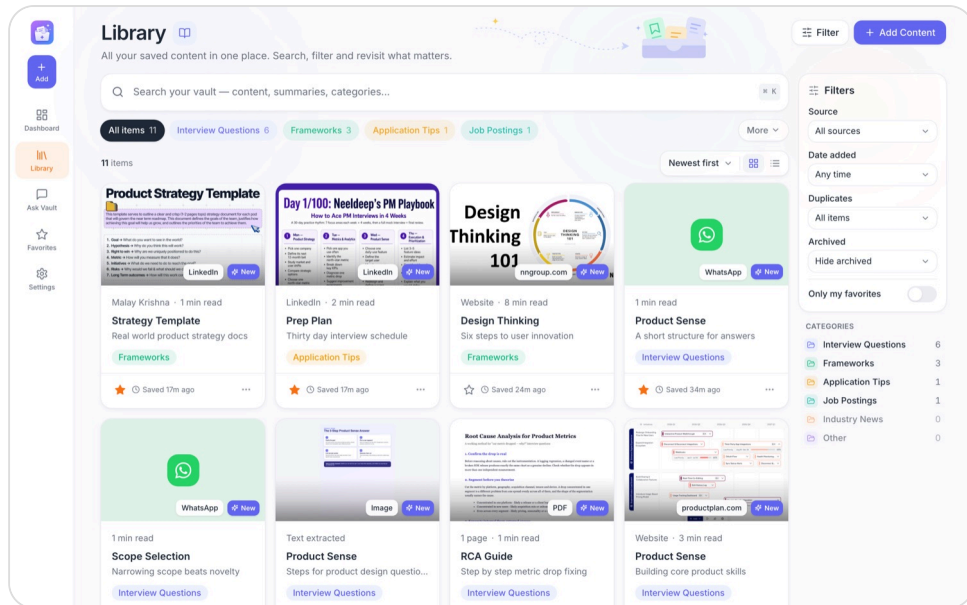
4

Check whether to trust the answer

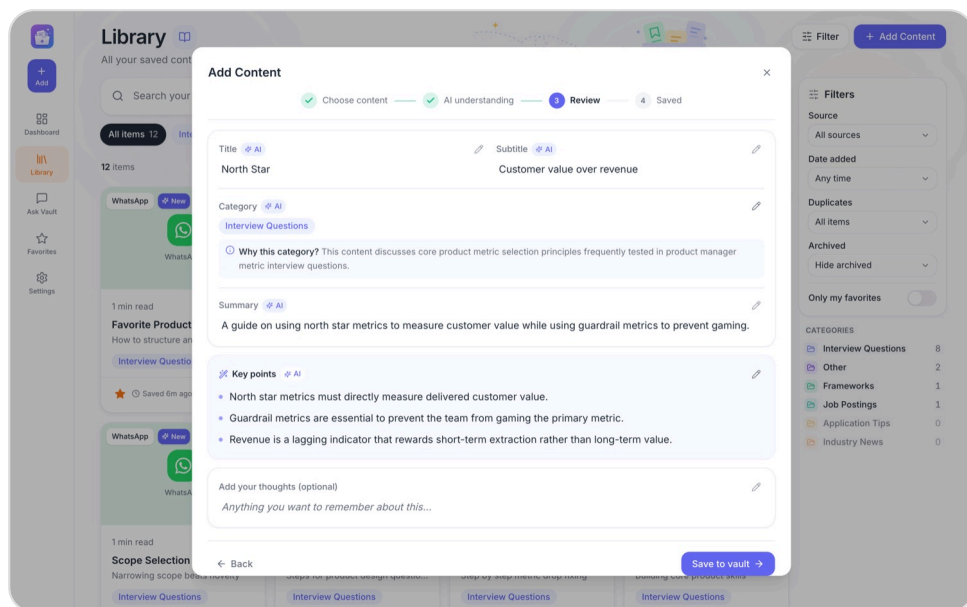
Every answer is numbered back to the saves it used.

Every claim traceable to a source

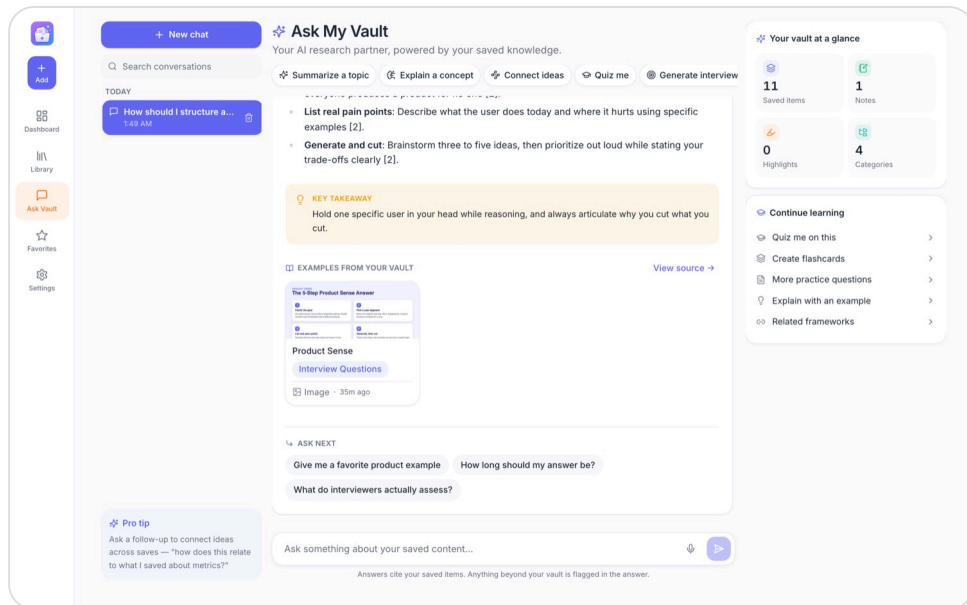
What it does



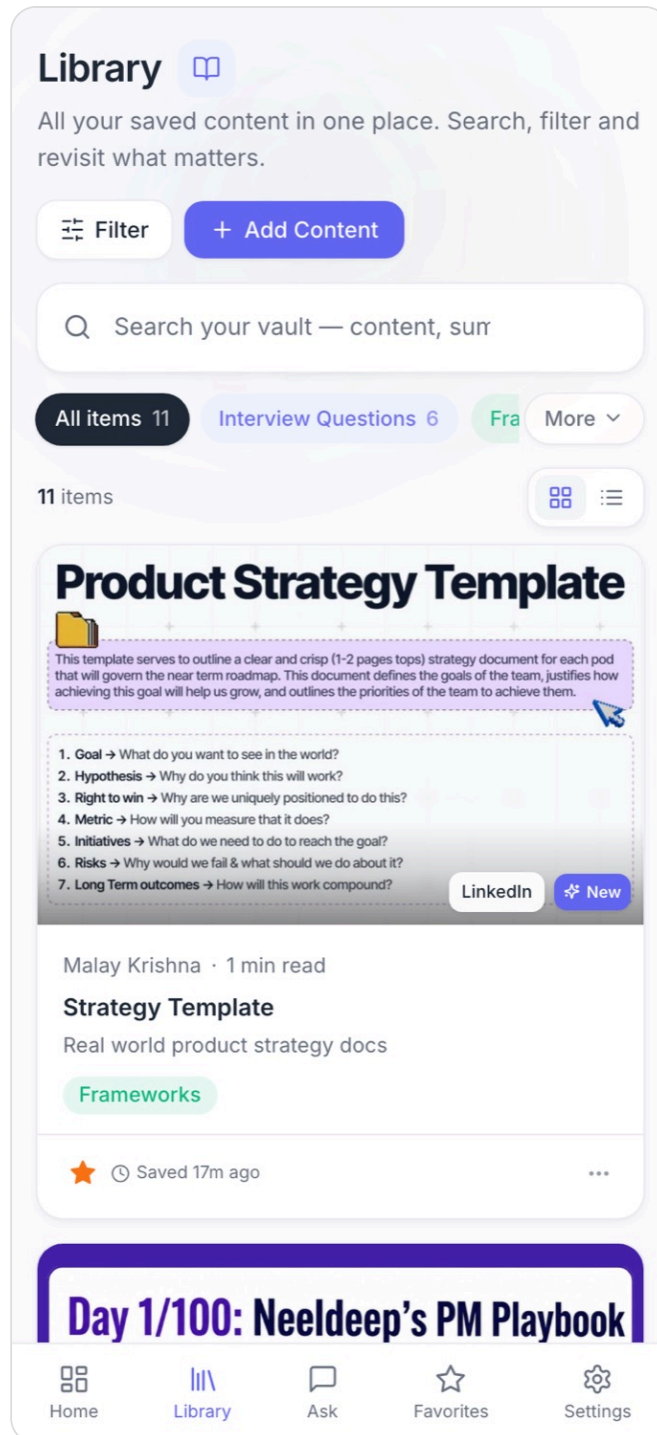
Every card knows what it is. A PDF shows its first page, a screenshot shows the image, a LinkedIn post shows the author. No generic file icons, because you should recognise a save without reading it.



The AI suggests, you approve. Adding something reads it, sorts it, summarises it and checks for duplicates, then stops and shows you the result. An **AI** chip marks what the model wrote and disappears once you edit it.



Answers you can check. Ask My Vault answers from your own saves. Open a source and you see the exact passage the answer used, not the item's summary.



Built for where saving actually happens. The sidebar becomes a bottom bar and cards stack into one column.

How it's organised

Five places, plus two decisions underneath.

Dashboard	: counts, recent saves, continue a conversation
Library	: everything, filtered by category / source / date / duplicates
↳ Item	: the content, its summary, highlights, related items
Ask My Vault	: conversations, each answer linked back to the Library
Favorites	: starred items, with study actions over them
Settings	: account, defaults, data

One message is one item

A WhatsApp export doesn't save as one item. Each message becomes its own, because a 40-message chat with three good ideas in it is otherwise one unsearchable blob. Splitting it up front is what lets a source link point at something small enough to check.

Two kinds of label, on purpose

	CATEGORIES	TAGS
Who sets them	The AI picks from a fixed list, and you can override it	You, freely
How many	One to three per item, never zero	Any
Why	A fixed list keeps filtering reliable. An open one gives you "Product Sense", "Product-Sense" and "PS" within a week	Your own structure, for what the list can't cover. Favourites are built as a tag

NEVER ZERO

Every item gets at least one category, falling back to `Other`. An "uncategorised" pile sounds harmless, but it's where things go to be forgotten, and forgetting is the problem this product exists to fix.

Decisions that shaped it

The feature list is the least interesting part. What got argued down is where the thinking is.

DECISION 01

Search by meaning, not keywords, and no model training

Keyword search fails on the query people actually have, because you remember a post's *idea*, not its words. Training a model has a different problem: your library changes daily, so every save means retraining. Search by meaning solves both, and every answer can point back at what it came from. That last part became the tiebreaker for every later call.

DECISION 02

Nothing is saved until you confirm

The normal way is to save the row first and tidy it in the background. I did the opposite: the AI work runs in memory, and only pressing save writes anything. An abandoned add leaves nothing behind, and a duplicate warning is only useful *before* you save. Telling someone they've just made a duplicate is a notification, not a feature.

DECISION 03

No LinkedIn scraping, even though it's the biggest source

Most of what I save is on LinkedIn, and auto-importing it would be the single most useful feature. It also breaks LinkedIn's terms, so the app only takes a pasted link or text. I wrote it into the brief as a locked rule before building, so I couldn't talk myself out of it at 1am when the manual flow felt slow.

DECISION 04

Cutting a feature I'd already planned

"Resurfacing" was feature #7 in my own brief: anything untouched for 14 days pops up in a reminder widget. Six of seven shipped. I cut this one, and not because it was hard. It was the cheapest thing left, two rules over a date. I cut it because **showing you something just because it's old is a guess about what you want, and there's no evidence for it**. It goes back on the list the day users show me otherwise.

DECISION 05

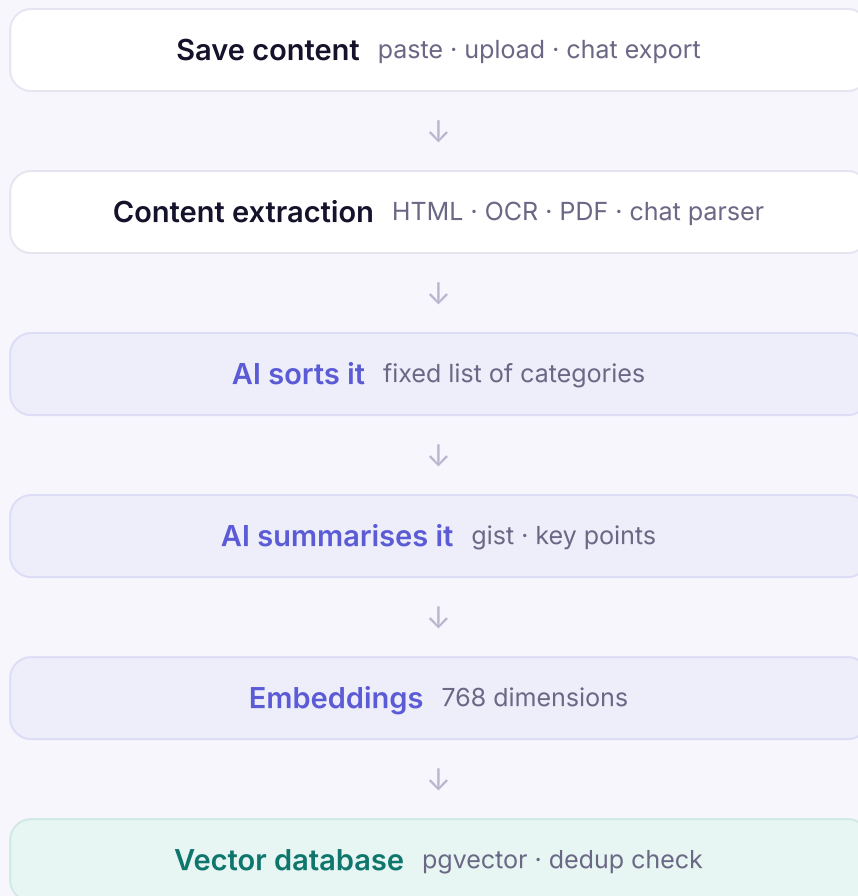
A hard \$0 budget, treated as a design constraint

Everything runs on a free tier, and text extraction happens in the browser where it costs nothing. That shaped the build, not just the bill. Uploads cap at 4MB because the hosting rejects anything bigger before my code runs, so a higher limit would swap a clear message for a confusing failure. A daily ping keeps the database awake, since free projects sleep and a portfolio link is most likely to be opened by the person you least want waiting.

AI architecture

Two AI paths, kept apart on purpose: one runs when content arrives, the other when a question is asked. They share an embedding model and nothing else.

INGESTION: WHEN CONTENT ARRIVES



RETRIEVAL: WHEN A QUESTION IS ASKED



What each stage does

- **Reading depends on the type.** Notes arrive as clean HTML, screenshots go through Tesseract in the browser, PDFs through `pdf-parse`, chat exports split one item per message.
- **Sorting and summarising happen in one call.** Gemini returns a category, title, summary and 3 to 5 key points. The reply is checked, not trusted: broken JSON gets repaired, and an empty reply throws instead of quietly saving a blank item.
- **The duplicate check reuses the same maths,** and runs *before* saving, while it can still change your mind.

- **Only sources the answer actually used get listed.** Search returns the top matches and the answer uses some. Listing them all would make "used 5 sources" a lie.

Designing for the model failing

The model is treated as unreliable, because it is.

WHAT GOES WRONG	WHAT HAPPENS
Model overloaded	Retry, then fall back to other models, cheapest first, since the best one allows about 20 requests a day free
Out of daily quota	A different message, because the advice differs: wait a minute versus wait until tomorrow
Broken reply format	Repaired field by field instead of thrown away
Sorting fails completely	The save still works, with blank fields you can fill in. The AI speeds this up, it doesn't block it
Search indexing fails	Retried. An unindexed item is invisible to search and skips the duplicate check, which is worse than a missing summary you can see

THE RULE UNDERNEATH

All of these fall back to the same thing: *you still get your content saved*. The one thing that must never slip is the source link. An answer you can't check is worse than no answer, because it looks exactly like one you can.

What broke, and what I took from it

Four failures worth writing down. Each changed how I think about shipping AI features.

– The app blamed the user for Google's outage

Saving started failing with *"The AI couldn't read this one."* The content was fine. Google's model was returning "busy, try later", and with one model, no retry and no logging, an outage at their end looked just like unreadable content.

Now: it retries, falls back to other models, logs every failure, and separates "the service is busy" from "you're out of quota".

Took from it: the error message is part of the feature. A wrong one sends someone off editing input that was never the problem, and if the system can't say which thing failed, that's a logging problem and not a wording one.

– A source link pointing at the wrong number

Sources are numbered by search rank, so an answer using only the fourth result says [4]. Reopening an old conversation renumbered them from 1, so [4] sat next to a card labelled 1. Worse, it showed the item's summary instead of the passage the answer used.

Now: the number and the exact passage are saved with the answer, and the item is still looked up fresh so edits show and deleted items drop out.

Took from it: a source link is a promise. Neither bug broke a test or looked wrong on screen. The bar isn't "looks cited". It's "can be checked".

– A bug only the live site would have found

Between reading your content and saving it, the text sits in memory. If that's missing the app re-reads it, but kept only the text and dropped the rest, so a note could save **with no body at all**. On my laptop it took a fifteen-minute pause to appear. On the live site, where the two steps can run on different machines, it's the normal case.

Took from it: deploying isn't just a new address, it's a different environment.

– Every uploaded file was public

Anyone with the link could open any PDF or screenshot, no login needed. Harmless on my laptop. Not harmless the moment the app had a public address.

Now: files are private and every link expires after two hours.

Took from it: fixing it took under an hour, and it was the thing I'd have been most embarrassed to be asked about. **The moment something is reachable, its weakest assumption becomes real.**

THE ONE THAT MATTERS MOST

The hard part of an AI product sits around the model, not inside it. The prompt took a day. Retries, fallbacks, honest error messages, checkable sources, and deciding what to do when the model returns nonsense took the rest of the week. That's the part that decides whether anyone trusts the output. The other thing I'd repeat: the constraints I wrote down as rules survived, and the preferences I only held in my head didn't.

What I'd measure, and what I'd ask

Set before launch, so success can't be defined afterwards to match the numbers. Each one names **the failure it would catch**, not just the thing it counts.

The four numbers I care about

QUESTION	MEASURE	WHY THIS ONE
Does saved content get used?	% of saves opened at least once in 30 days	The main claim. If this doesn't beat roughly zero, nothing else matters.
Does search actually work?	% of questions where someone opens a source	An answer nobody clicks into is just a nice-sounding answer. The failure I fear most.
Is it faster than before?	Time from question to the right save	It only earns its place if it beats scrolling LinkedIn saves.
Is the AI trusted?	% of AI-filled fields edited before saving	Lots of edits means it's getting things wrong. Zero probably means nobody's reading them.

The five questions I'd ask users

1. Show me the last five things you saved. Where are they? (*Is "scattered" real?*)
2. When did you last go looking for something you'd saved? What did you do?
3. What happened when you couldn't find it? (*Googling it again means the save was wasted.*)
4. Here's an answer built from three of your saves. Do you trust it? What would make you check?
5. You haven't opened this in three weeks. Should the app have said so? (*Decides whether resurfacing comes back.*)

WHAT WOULD CHANGE MY MIND

If three of five describe hunting for something they knew they'd saved and giving up, **resurfacing goes back in**, triggered by what they were looking for and not by how old the item is. If nobody opens a source, the trust model is wrong.

How it's built

Frontend	React (Vite) + Tailwind, hosted on Vercel
API	Express on Vercel, same domain as the site
Database	Supabase Postgres with <code>pgvector</code> for search by meaning
Login	Supabase Auth, Google and email/password. Every table locked to its owner
AI	Gemini for sorting, summaries and answers, <code>gemini-embedding-001</code> for search
In the browser	Tesseract.js reads screenshots, pdf.js renders PDF covers. Both free, both off the server

Two choices worth calling out: the API sits on the same domain as the site, so there are no cross-domain requests in production, and text extraction runs in the browser, off the server and off the bill.

Product Requirements: MVP

The brief the build ran against, written before the first line of code.

1. Problem

Saved learning material scatters across LinkedIn, group chats and bookmarks and rarely gets opened again (full version above). What it means for the build: summaries are written at save time instead of on demand, and "find the right thing fast" matters more than "store everything".

2. Who it's for

MVP: people learning product management, whose content is predictable enough to make a fixed category list work. **Long term:** anyone who saves things to learn from. **Not yet:** other subjects, which is a categories job and not a rebuild (reasoning above).

3. Goals and non-goals

GOALS	NON-GOALS
One place that takes every format this content arrives in	A general read-later app
Judge if an item is worth reading, without reading it	Replacing LinkedIn or WhatsApp
Answer questions across everything saved, with sources	Answering from general knowledge, not your saves
Catch duplicates before they pile up	Sharing or multi-user features

4. Rules locked before building

- **No LinkedIn scraping.** Paste a link or the text only, since scraping breaks their terms.
- **No listening to WhatsApp.** You export a chat yourself and upload the `.txt`, which gets split into items.
- **Single user for now.** No sharing, but every table has an owner from day one so multi-user isn't a rewrite.
- **Resurfacing cut** (2026-08-06, after the other six shipped). See Decision 04.

5. Feature scope

#	FEATURE	STATUS
1	By hand: text, link, image (OCR), PDF, WhatsApp export, question, job posting	Shipped
2	Sorting and summarising at save time	Shipped
3	Embeddings in pgvector, so search works by meaning	Shipped
4	Library: list, filter, search, sort	Shipped
5	Ask My Vault, every answer linked back to the saves it used	Shipped
6	Duplicate warnings when a new save is too close to an old one	Shipped
7	Resurfacing widget	Cut , see Decision 04

6. Data model

TABLE	WHAT IT HOLDS
items	The content, its summary, key points, category, dates, and fields like page count
embeddings	The maths behind search by meaning, one per item
item_categories, tags	Categories and tags. Favourites are stored as a tag
duplicates	Near-identical pairs and how close they were
annotations	Highlights and notes, anchored so they survive edits
chat_sessions, chat_queries	Conversations, answers, and the sources each one used

7. Categories

Interview Questions (Product Sense · RCA · Metrics · Strategy · Behavioural) · Job Postings · Application Tips · Frameworks · Industry News · Other.

8. What this project demonstrates

Defining a problem and rejecting the obvious answer, scoping an MVP with locked rules and one planned feature cut, building the search pipeline end to end, designing for trust and for the model failing, shipping it alone on a \$0 stack.

9. Roadmap

None of this is built. Each says why it isn't in the MVP.

Next: what still gets in the way

WHAT	WHY IT MATTERS
Share straight from any app	Every save today is copy, switch app, paste. Cutting that step matters more than anything else here, because the whole idea rests on saving staying easy.
A first-run walkthrough	A new account opens onto an empty vault. It should land one real item and one answered question before asking anyone for trust.
Export the vault	What's saved here can only be read here. It belongs inside the reset and delete flows too, since a download before wiping a library makes that undoable.
Browser extension	The desktop half of the same problem.

Then: past the first subject

WHAT	WHY IT MATTERS
Any job, any subject	Only the category list is specific to product management. Make it configurable, or infer it from what someone saves, and this becomes a general study tool.
Practice mode	The saves already hold questions and worked answers. Turning those into drills makes this a study tool instead of a library, and it's the best reason to come back.
Collections	Grouping saves into a plan ("week 3: metrics") gives the vault a shape a flat list can't.

Later: needs other people in it

WHAT	WHY IT MATTERS
Shared vaults	Study groups already pass this material around in chats. A shared vault is where export and multi-user meet.
Anonymous "what's trending"	The one network effect here: the embeddings that power personal search would group across accounts to show what everyone is studying. Needs multi-user and enough people to mean anything.

WHY IN THIS ORDER

Next fixes what annoys people about something that already works. **Then** widens who it works for. **Later** needs other people signed up, so it can't be pulled forward however tempting.

PM Content Vault: built and shipped as a solo project, 2026.

Live app (<https://pm-content-vault.vercel.app>) · **Source** (<https://github.com/RADHIKA2909/pm-content-vault>) · **Demo**
login `demo@pmvault.app` / `PMVaultDemo2026!`